Testing an MCP Server

> Part of the MCP Server curriculum. This document covers unit-testing tool functions directly, integration-testing through a real client, fixture data patterns, a class of entry-point bug only a real subprocess launch can catch, and registry-wide tests for rejection rules.

1. What's testable, and at what level

- **Unit tests, against the plain function.** Strip away the `@mcp.tool()` decorator mentally, and most tools are just ordinary Python functions — test them exactly like any other function, with the same pytest fundamentals used for a backend API.
- **Integration tests, through a real MCP client.** A unit test on the underlying function proves the *logic* is right, but it can't catch a bug in how the function got registered as a tool, how its schema was generated, or how the protocol layer serializes its result — those need an actual client issuing an actual `tools/call` against a running (or subprocess-launched) server instance.

Both levels matter, for the same reason both matter for a REST API: a unit test is fast and precise about logic; an integration test is what actually proves the *contract* — the thing a real caller depends on — works end to end.

2. Fixture data pattern

If the server reads a data source directly (a file, a local database), reuse the same fixture-CSV-and-monkeypatch pattern described in the API backend curriculum: small, hand-built fixture data written to a temporary directory per test, with the one configuration point the loading code reads from monkeypatched to point at it — never test against real, large, or gitignored production data.

If the server instead wraps a REST API (the common shape from Tool Design, §2's direct-wrap pattern), point its base URL configuration at a locally-running test instance of that API for integration tests, rather than mocking the HTTP client itself — the same reasoning as the backend curriculum's monkeypatch-over-mocking-your-own-code preference: mocking the HTTP call would skip over exactly the request-shaping and response-handling code you actually want under test.

3. Testing argument-resolution guards

The four-case ladder from Tool Design, §3 maps directly onto a parametrized test:

```python
import pytest

@pytest.mark.parametrize(
    ("input_value", "expect_error", "expect_resolved_to"),
    [
        ("Canonical Name", False, "Canonical Name"),   # exact match
        ("Canonicl Nam", False, "Canonical Name"),      # fuzzy match, resolves silently
        ("Xyzzy", True, None),                          # below threshold, explicit error
        ("", True, None),                               # empty/malformed input
    ],
)
def test_resolve_identifier(input_value, expect_error, expect_resolved_to):
    result = resolve_identifier(input_value, candidates=CANONICAL_NAMES)
    if expect_error:
        assert result.is_error
    else:
        assert result.resolved == expect_resolved_to
```

Write at least one test asserting the error case's message actually contains a useful candidate suggestion, not just that it errors — the *content* of the error is the entire point of that branch (a model can't act on an error that just says "not found").

4. Testing response shaping

Assert two things independently, since they're easy to get subtly wrong relative to each other:

1. **Trimming triggers exactly when documented** — an explicit selection list returns everything requested, uncapped; an omitted/default selection is capped.
2. **The annotation field is present exactly when trimming actually happened** — not present on the uncapped path, present on the capped one. This is an easy off-by-one-condition bug (e.g. accidentally attaching the note whenever the *tool* has a cap configured, rather than only when a response actually exceeded it) that silently defeats the "no note means complete" contract from Tool Design, §4 without any obviously broken behavior to notice.

5. A subtle failure class: process/module identity bugs

If a server can be launched more than one way (directly running its module file vs. running it as a package), watch for a specific, easy-to-introduce bug: a module gets imported twice under two different qualified names during startup, producing **two separate instances of what was meant to be one shared object** — for example, a `tools` module that registers itself onto a server object it imports from its parent package, where an import-order quirk causes the parent package to load twice, once under each name. The tools end up registered on the *other* instance from the one your actual entry point runs, so the server starts successfully, looks healthy, and simply has none of its tools.

This class of bug is **invisible to an in-process unit test** — importing the modules directly in a test process doesn't reproduce the double-import condition an external subprocess launch triggers. The only real way to catch it is a regression test that actually launches the server the way a real client does:

```
import subprocess

def test_entry_point_registers_tools():
    proc = subprocess.run(
        ["python", "-m", "my_package"],
        input=INITIALIZE_AND_LIST_TOOLS_REQUEST,
        capture_output=True, text=True, timeout=10,
    )
    assert "get_widget" in proc.stdout
```

The general lesson: **if an entry point can be invoked more than one way, test the specific way it's actually invoked in production** — an in-process import proves the code is syntactically fine, not that the real launch path produces the object graph you expect.

6. Testing a broken-tool-module-at-import-time failure mode

Decide deliberately what should happen if one tool module fails to import at startup (a syntax error, a bad dependency) — should the whole server refuse to start, or should it start anyway with that module's tools simply missing? Both are legitimate choices depending on how the server is operated (a server run interactively, where a shrunk tool list is immediately visible to whoever's testing it by hand, can reasonably tolerate the softer failure; a server run unattended, where nobody would notice a quietly-missing tool, generally shouldn't). Whichever you pick, write a test that deliberately breaks one tool module's import and asserts the chosen behavior actually happens — don't leave this as an unverified assumption about how your framework's import machinery behaves.

7. Testing the rules that reject, in every tool that has them

The guards in Tool Design, §6.3 are only as good as their coverage. The recurring bug is a rule implemented in the first three tools that take a list and forgotten in the fourth. Write the test **parametrized over the tool registry**, not over a hand-picked list: for every registered tool with a list-typed argument, assert that passing an explicit empty list returns a structured error. A new tool is then covered automatically the day it is added — and if it cannot be called that way, the test tells you why.

The same pattern covers the other conventions: every tool in a family returns the envelope fields unchanged (assert `note`, `caveats`, `attribution` equal the upstream's), every `summary` carries a `unit`, and every user-visible string passes the typography normaliser. Feed the tools fixtures that include the awkward cases — a series that ends early, a combination the upstream refuses (422), an entity the upstream has but the downstream picker does not — and assert the *tool-level* message, not the upstream's.

Finally, remember that **this server's tests are not the only ones its change can break**: the consuming agent's suite exercises the real tool list, so a new or renamed tool turns the agent's suite red even when this one is green. Run both (Agent Testing, §10).

8. Core libraries summary

Library	Used for
pytest	test runner, fixtures, parametrization — same as the API backend curriculum
`subprocess` (standard library)	launching the server as a real, separate process for entry-point and protocol-level integration tests
A test-only instance of whatever the server wraps	integration tests against real request/response shaping, without touching production data

See also

- MCP Server curriculum index
- Tool Design — the guards and shaping rules this document's tests verify
- API Backend Testing — the fixture and mocking patterns this document builds on
- Deployment — running the tested server for real